

Lecture: 6
Linked List (Part-II)

A data structure is how data is stored and organised.

Then, depending on how the data is stored and organised, different operations can be applied to the data.

What differentiates data structures from each other can either be how the data is stored and organised, what can be done with and/or to the data, or both.

A bulk of the problems in computer science requires some sort of data, and our need to efficiently retrieve information from the data depending on the problem type is what motivates the creation of the different data structures that we have today.

For example, a linked list stores and organises data in a linear order with the help of pointers (this is the “how data is organised”), and these pointers allow us to easily insert and remove elements without reorganising the entire data structure (this is the “what can be done”). This makes a linked list a linear data structure (as opposed to a non-linear one).

Traversing a Linked List:

Traversing means going through every single node, starting with the head of the linked list and ending on the node that has a next value of None.

Traversing just means iterating.

Algorithm for traversing a linked list:

Let LIST be a linked list in memory. This algorithm traverses LIST , applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

Step:1) Set PTR := START. [Initializes pointer PTR]

Step:2) Repeat Steps 3 and 4 while PTR != NULL.

Step:3) Apply PROCESS to INFO[PTR].

Step:4) Set PTR := LINK[PTR]. [PTR now points to the next node]

[End of Step 2 loop.]

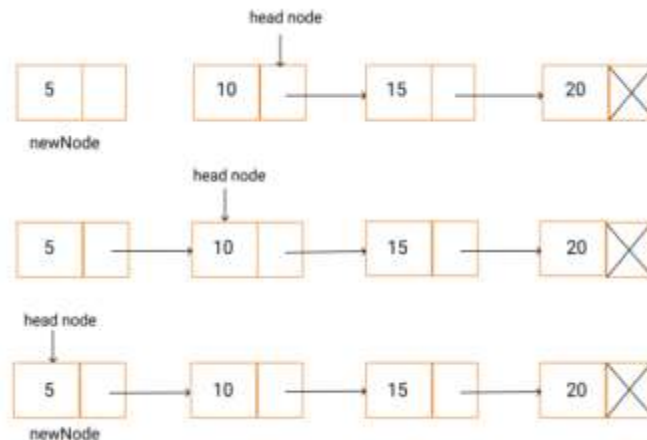
Step: 5) Exit

Insertion into a Linked List:

Insertion in Linked List can happen at various places:

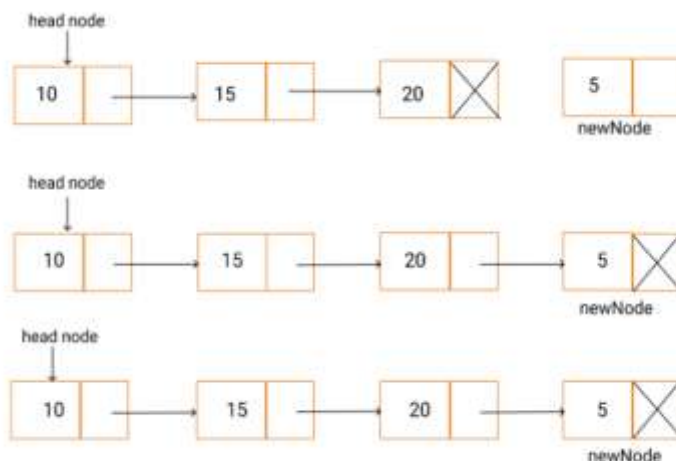
- At the beginning of the linked list
- at the end of the linked list
- at a given position in the linked list
- at the middle of the linked list

Insertion at the beginning of the Linked List:



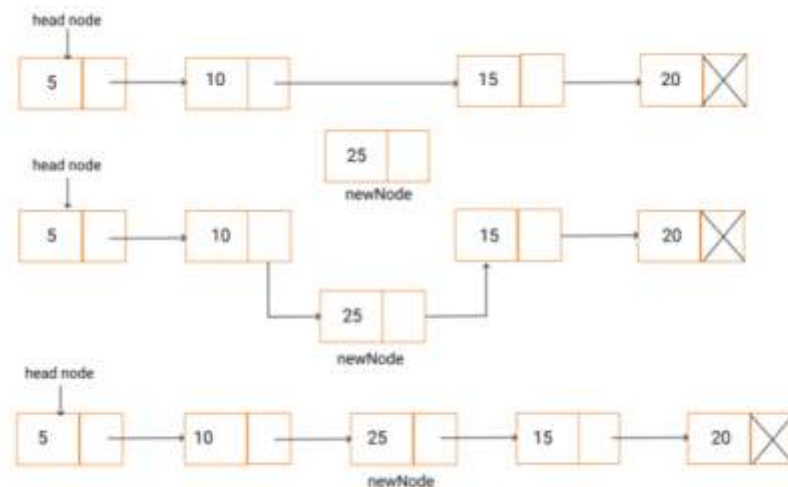
- Let us assume a newNode as shown above. The newNode with data=5 has to be inserted at the beginning of the linked list.
- For this to happen, the address part of the newNode should point to the address of the head node. Once it points to the head node, then the newNode is made as the head node as shown in the figure.

Insertion at the end of the Linked List:



- Let the newNode with data=5 is to be inserted at the end of the linked list.
- Now, start traversing the list and continue traversing until the address part of the node is NULL.
- Once the address part of a node is found to be NULL, then make the address part of that node to point to the address of the newNode and the address part of the newNode is referenced to NULL. Hence it becomes the last node.

Insertion at a given position in the linked list:



- Let us assume that a newNode with data=25 has to be inserted at the 2nd position in the linked list.
- Start traversing the list from the head and move up to the position-1. In this case, we want to insert at 2nd position, you need to traverse till (2-1)=1.
- Once you reach the position-1 node, the address part of the newNode is set to the address contained by the position-1 node.
- Now, the address part of the position-1 node is made to point to the newNode.